PIXORAMED

Documento de entrega

Workflows BPMN 2.0, justificativa técnica e análise de ética, governança e segurança do protótipo de classificação de imagens de câncer.

> Protótipo acadêmico. O PixoraMed não é um dispositivo médico, não foi validado clinicamente e não faz diagnóstico.

Programa	Bootcamp de IA II/2026, TCS e SENAI Londrina
Grupo	Henrique Pereira da Silva, João Pedro Gonçalves Ferreira, Ramon Camilo Figueiredo, Samuel Rosa Costa
Orientador	Prof. André Maurício dos Santos
Banca	23/11/2026
Sistema	https://pixoramed.vercel.app

Sumário

1. Contexto, problema e objetivo
2. Workflows BPMN 2.0
    1. Login e cadastro
    2. Microsserviço de IA
    3. Uso do sistema: análise de uma imagem
3. Justificativa técnica
    1. Arquitetura
    2. Escolha das tecnologias
    3. Dados: Python, Pandas e SQL
    4. Modelos de machine learning
    5. Resultados medidos
    6. Infraestrutura: Docker e Terraform
    7. Qualidade e testes
4. Ética, governança e segurança
    1. Uso responsável e limites do protótipo
    2. Viés e vazamento de dados
    3. Privacidade (LGPD)
    4. Segurança da aplicação
    5. Licenças e créditos dos dados
    6. Riscos e mitigações
5. Conclusão e trabalhos futuros
6. Referências

1. Contexto, problema e objetivo

A leitura de imagens médicas é uma etapa demorada da triagem de câncer, e a fila de exames depende da disponibilidade de especialistas. Redes neurais treinadas com imagens públicas conseguem separar padrões visuais de algumas lesões, mas um número alto de acurácia sozinho não diz se o modelo serve para pacientes reais.

O **PixoraMed** é um protótipo que classifica imagens de três tipos de câncer e mostra, no mesmo site, o resultado de cada análise e o desempenho medido de cada modelo, com os limites dessa medida:

Módulo	Tipo de imagem	Classes	Dataset
Pele	Dermatoscopia, colorida	benigno, maligno	HAM10000
Oral	Histopatologia, colorida	normal, carcinoma espinocelular (SCC)	Multi Cancer
Cérebro	Ressonância magnética, tons de cinza	glioma, meningioma, hipofisário	Multi Cancer

Objetivo

Entregar um fluxo completo e funcional, do treino dos modelos à interface, que mostre como um sistema de apoio à triagem poderia funcionar e, ao mesmo tempo, deixe claro onde ele falha. O sistema permite:

- criar conta e entrar com login seguro;
- escolher o módulo, enviar uma imagem e receber a classe prevista com as probabilidades de cada classe;
- consultar o histórico das próprias análises e exportá-lo em CSV;
- comparar os resultados dos treinos (acurácia, F1, desvio entre folds) e ler as limitações de cada modelo.

Público e valor

O cenário pensado é o de um médico que já sabe qual exame tem em mãos e quer uma segunda leitura rápida para priorizar casos. O ganho de tempo é hipotético: os datasets usados têm licença não comercial, o protótipo não foi validado clinicamente e por isso não calculamos retorno financeiro.

> Escopo deliberadamente limitado a três módulos. Outros cânceres testados nos notebooks (por exemplo, mama) ficaram fora do sistema por resultados ruins em imagens de outra fonte (seção 3.5).

2. Workflows BPMN 2.0

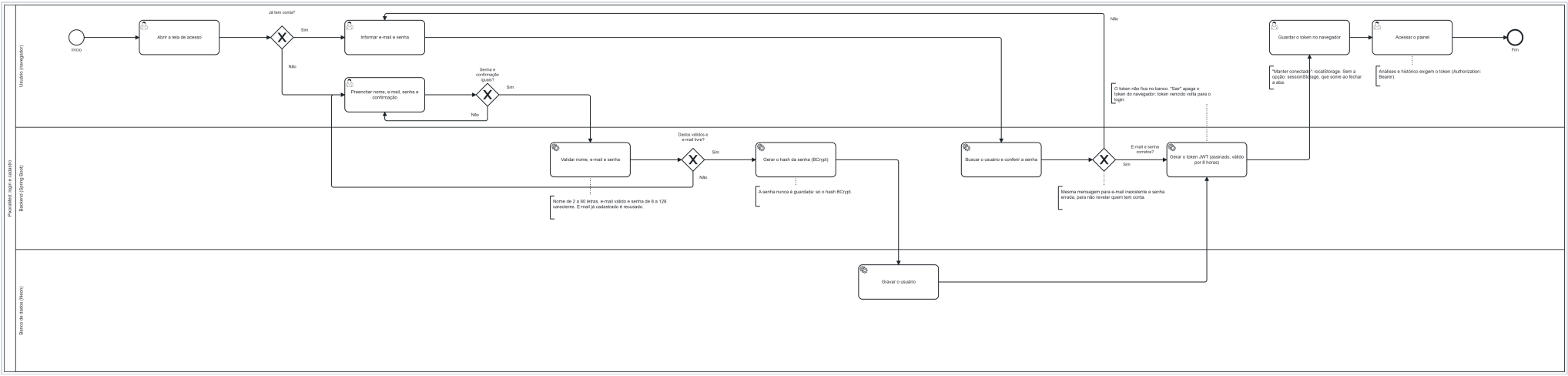
Os três processos foram modelados em BPMN 2.0 e estão em `docs/bpmn/` como arquivos `.bpmn`, que abrem no bpmn.io e no Camunda Modeler, e como imagens `.png`. Os diagramas são gerados por código (`docs/bpmn/generator/generate.py`), o que mantém as etapas iguais ao que o sistema faz de verdade: quando o código muda, o processo é atualizado e exportado de novo.

Workflow	Raias	O que mostra
1. Login e cadastro	Usuário, Backend, Banco	Cadastro com validação, hash BCrypt da senha e emissão do token JWT
2. Microsserviço de IA	Backend, ai-service	Subida do serviço, token interno, validações e quem decide a classe
3. Uso do sistema	Usuário, Sistema, IA	Uma análise completa, do módulo escolhido até o resultado com avisos

O workflow 3 começa com o usuário já logado, como pedido pelo orientador; o login fica todo no workflow 1. As anotações de texto dentro dos diagramas registram as regras de negócio (limites, mensagens de erro, o que é ou não gravado), para que o processo possa ser lido sem o código.

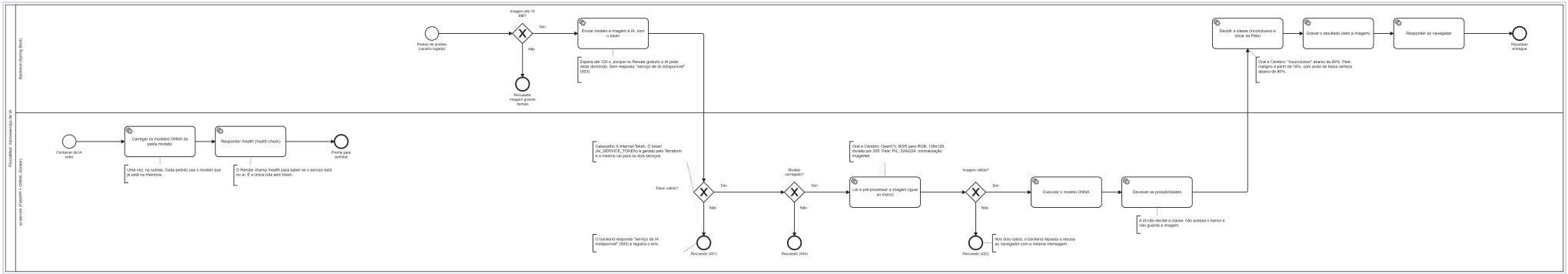
As próximas três páginas trazem cada diagrama em página horizontal, seguido das decisões que ele documenta.

2.1 Login e cadastro



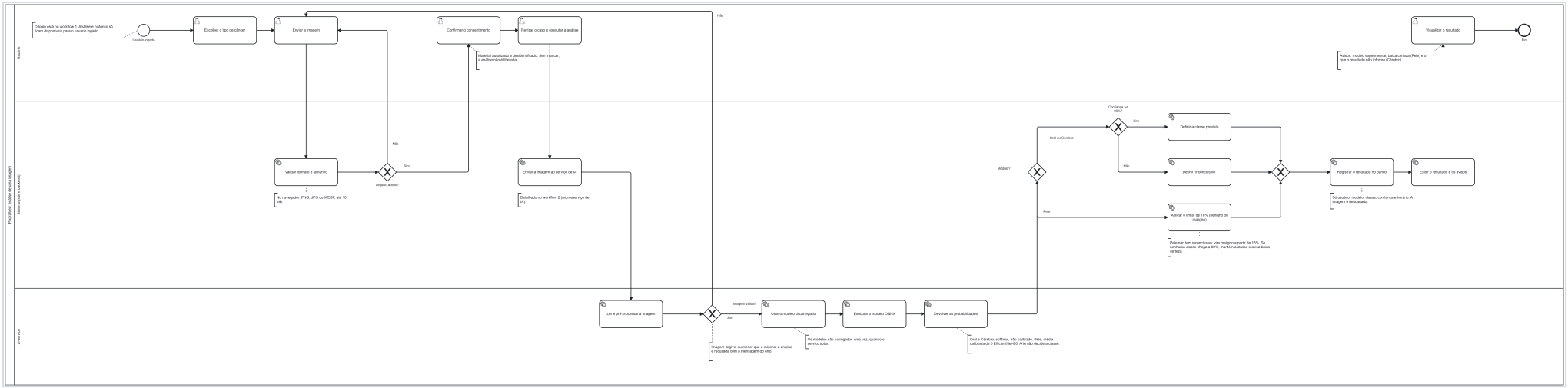
- O navegador confere se a senha e a confirmação são iguais; o backend valida de novo nome (2 a 80 letras), e-mail e senha (8 a 128 caracteres), porque validação no navegador pode ser contornada.
- A senha nunca é gravada: o banco guarda só o hash BCrypt.
- E-mail inexistente e senha errada recebem a mesma mensagem, para não revelar quem tem conta.
- O token JWT é assinado pelo backend e vale 8 horas. Ele não fica no banco: "Sair" apaga o token do navegador e um token vencido volta para o login.
- "Manter conectado" guarda o token no `localStorage`; sem a opção, no `sessionStorage`, que some ao fechar a aba.
- Análises e histórico exigem o cabeçalho `Authorization: Bearer`.

2.2 Microsserviço de IA



- Os modelos ONNX são carregados uma vez, quando o container sobe; cada pedido usa o modelo que já está na memória.
- `/health` é a única rota sem token, usada pelo Render para saber se o serviço está no ar.
- O backend recusa imagens acima de 10 MB antes de chamar a IA e envia o cabeçalho `X-Internal-Token`. Token inválido: 401; modelo desconhecido: 404; imagem ilegível: 422.
- O pré-processamento repete o do treino: Oral e Cérebro com OpenCV (BGR para RGB, 128×128, divisão por 255); Pele com PIL (224×224, normalização ImageNet).
- A IA só devolve probabilidades. Ela não decide a classe, não acessa o banco e não guarda a imagem.
- O backend espera até 120 s, porque no plano gratuito a IA pode estar dormindo; sem resposta, mostra "serviço de IA indisponível" (503).

2.3 Uso do sistema: análise de uma imagem



- O navegador aceita PNG, JPG ou WEBP de até 10 MB.
- O usuário confirma que o material é autorizado e desidentificado; sem marcar, a análise não é liberada.
- Oral e Cérebro: abaixo de 80% de confiança, o resultado é "inconclusivo".
- Pele: não há inconclusivo. A classe é "maligno" a partir de 18% de probabilidade calibrada; se nenhuma classe chega a 80%, a tela mantém a classe e mostra "baixa certeza".
- O banco registra só usuário, modelo, classe, confiança e horário; a imagem é descartada.
- A tela de resultado mostra os avisos: modelo experimental, baixa certeza (Pele) e o que o resultado não informa (Cérebro não detecta ausência de tumor).

3. Justificativa técnica

3.1 Arquitetura

O sistema tem três serviços independentes, cada um na própria pasta do repositório, com Dockerfile, README, arquivo `.env.example` e documentação de arquitetura:

Navegador: frontend React servido pela Vercel. Chama sempre `/api` no próprio domínio.

↓ rewrite `/api/*` configurado no `vercel.json`

backend (Spring Boot, Render): login JWT, histórico, importação dos treinos e decisão da classe final.

SQL →

PostgreSQL (Neon): tabelas `users`, `trainings` e `predictions`.

↓ HTTP com o cabeçalho `X-Internal-Token`

ai-service (FastAPI + ONNX Runtime, Render): pré-processa, executa o modelo e devolve as probabilidades. Sem banco e sem guardar imagem.

Por que separar a IA do backend. Os modelos e as bibliotecas de visão (OpenCV, ONNX Runtime) somam quase toda a memória e o tamanho da imagem Docker. Num serviço próprio, a IA pode ser atualizada, escalada ou trocada sem mexer no login nem no banco, e não tem acesso a nenhum dado de usuário: recebe uma imagem e devolve números.

Por que a regra de decisão fica no backend. Limiar de inconclusivo, limiar da Pele e o que é gravado são regras de negócio, que mudam com mais frequência que o modelo. Ficando no backend, elas são configuráveis por variável de ambiente (`INCONCLUSIVE_THRESHOLD`) e cobertas por testes automatizados, e o frontend não tem regra nenhuma nem segredo nenhum.

Por que o site chama `/api` no próprio domínio. O rewrite da Vercel repassa as chamadas ao backend. O navegador não precisa saber onde o backend está, não há variável de ambiente com endereço no frontend, e o mesmo código funciona no desenvolvimento local, onde o Vite faz o repasse.

3.2 Escolha das tecnologias

Camada	Tecnologia	Motivo
Frontend	React 19, Vite, React Router, TanStack Query, Recharts	Componentes reaproveitados entre as telas; TanStack Query cuida de cache, carregamento e erro das chamadas; telas internas carregadas sob demanda (pacote principal de 774 KB para 320 KB).
Backend	Java 21, Spring Boot 4, Spring Data JPA, Spring Security, Maven	Padrão de mercado para APIs com camadas (controller, service, repository); Spring Security já traz BCrypt e validação de JWT, sem código de criptografia próprio.
IA	Python 3.12, FastAPI, ONNX Runtime, OpenCV, Pillow	Os modelos foram treinados em PyTorch e exportados para ONNX: o serviço roda sem o PyTorch, com imagem menor e inferência rápida em CPU.
Banco	PostgreSQL (Neon na nuvem, container no local)	Banco relacional para dados estruturados (usuários, treinos, análises); Neon tem plano gratuito sem cartão.
Infra	Docker Compose, Terraform	Compose sobe os quatro serviços no notebook com um comando; Terraform descreve a nuvem como código (seção 3.6).
Hospedagem	Vercel, Render, Neon	Planos gratuitos que aceitam Docker e deploy automático a cada push na branch `main`.

3.3 Dados: Python, Pandas e SQL

- **Python e Pandas** ficam nos notebooks de treino (repositório Projeto-ML-Cancer): leitura dos metadados, separação estratificada em 5 folds, cálculo das métricas por fold e geração da tabela de resultados dos treinos. O sistema em produção não usa Pandas.
- **A tabela de resultados** (`training-results.csv`, 29 execuções) entra no backend e é importada para a tabela `trainings` toda vez que o backend sobe. A importação é idempotente: compara pela chave data/hora + dataset e insere só as linhas novas.
- **SQL**: o PostgreSQL guarda usuários, treinos e análises. O Spring Data JPA gera as consultas (por exemplo, o histórico do usuário ordenado por data, com limite de 100 linhas), e as tabelas são criadas pelo Hibernate a partir das entidades.
- **Visualização**: o site mostra gráficos de barras com o desvio entre folds na página de treinos, um gráfico de rosca com as probabilidades de cada classe no resultado e as figuras do treino da Pele (curva ROC, matriz de confusão, calibração).

3.4 Modelos de machine learning

Módulo	Modelo	Entrada	Decisão
Pele	Ensemble de 5 EfficientNet-B0, calibração de Platt (~80 MB)	224×224, cor, normalização ImageNet	maligno a partir de 18%; aviso abaixo de 80%
Oral	CNN pequena, ~24 mil parâmetros (~110 KB)	128×128, cor, /255	inconclusivo abaixo de 80%
Cérebro	CNN pequena, ~24 mil parâmetros (~110 KB)	128×128, cinza, /255	inconclusivo abaixo de 80%

A CNN de Oral e Cérebro tem 3 blocos convolucionais (16, 32 e 64 filtros), pooling adaptativo, dropout de 0,25 e uma camada linear; treino com Adam (taxa 0,001), lote 64 e validação cruzada estratificada em 5 folds com semente fixa. Uma rede pequena foi suficiente nesses dois datasets e roda em ~0,1 a 0,2 s numa fração de CPU.

Na Pele, o limiar baixo (18%) foi escolhido para errar menos casos malignos, ao custo de mais alarmes falsos, o que faz sentido numa triagem. Na Oral e no Cérebro, o limiar de 80% troca algumas respostas por "inconclusivo" para reduzir erros de baixa confiança.

> O pré-processamento precisa ser idêntico ao do treino. Num teste, ler a imagem em BGR ou tirar a divisão por 255 derrubou a acurácia da Oral de 96,5% para cerca de 52%. Por isso o ai-service repete exatamente as etapas dos notebooks, e o workflow 2 documenta essas etapas.

3.5 Resultados medidos

Módulo	Teste	Resultado	Observação
Oral	200 imagens separadas, mesma fonte	96,5% (193/200)	Com o limiar: 188 conclusivas, 98,4% de acerto
Cérebro	300 imagens separadas, mesma fonte	98,3% (295/300)	Com o limiar: 297 conclusivas, 99,0% de acerto
Cérebro	5 folds, 1.000 imagens por classe	95,17% ± 0,55	Cinza sem normalização (a melhor das 4 configurações)
Pele	Reprodução do treino, 1.915 imagens, divisão por lesão	AUC 0,939	IC 0,922–0,954; sensibilidade 89,0%; especificidade 83,7%; VPP 51,2%
Pele	Modelo do app, 21 imagens internas	8/8 malignos, 7/13 benignos	Os 6 alarmes falsos receberam aviso; amostra pequena

Como ler esses números

- **"Mesma fonte" não é teste externo.** As imagens separadas de Oral e Cérebro vêm do mesmo dataset do treino. Elas medem se o modelo aprendeu o dataset, não se funciona em outro hospital.
- **Evidência de mudança de domínio.** Um modelo de mama treinado no mesmo processo teve 96,6% na mesma fonte e 34,5% em 29 imagens de outra fonte, e 19 desses erros passaram pelo limiar de 80%. Por isso o módulo foi retirado do sistema.
- **Pele: reprodução não é o modelo do app.** O código original do ensemble foi perdido; os números de AUC, sensibilidade e especificidade vêm de uma reprodução do treino. Na reprodução, a acurácia

(84,5%) quase empata com sempre responder "benigno" (83,9%), por isso a AUC e a sensibilidade são as métricas relevantes.

- **Confiança não calibrada.** Em Oral e Cérebro, a "confiança" é a saída softmax da rede, não uma probabilidade calibrada de doença.

Tempo por previsão no ai-service (container de 512 MB, 0,1 CPU): Cérebro 0,1 s, Oral 0,2 s, Pele 1,7 s.

3.6 Infraestrutura: Docker e Terraform

- **Docker**: backend e ai-service têm Dockerfile próprio, rodam com usuário sem privilégios e têm health check. O backend usa build em duas etapas (Maven + JDK para compilar, só o JRE para rodar) e limita a memória da JVM a 70% do container.
- **Docker Compose** (`infra/docker-compose.yml`): ambiente local com banco, IA, backend e frontend numa rede interna. As portas abrem só em `127.0.0.1`, o ai-service não expõe porta e cada serviço espera o anterior ficar saudável. Serve também de plano B se a internet falhar na apresentação.
- **Terraform** (`infra/terraform`): descreve o projeto da Vercel, os dois serviços do Render com suas variáveis e o banco do Neon, com versões de provider fixas. O banco tem `prevent_destroy`, que impede apagá-lo por engano. O token interno e o segredo do JWT são gerados pelo próprio Terraform, e nenhum valor secreto está escrito nos arquivos `.tf`.

3.7 Qualidade e testes

- Backend: 5 testes automatizados (regra de decisão da classe e leitura do CSV real de treinos).
- ai-service: 20 testes automatizados (rotas, token interno, pré-processamento e respostas de erro).
- Frontend: ESLint sem erros nem avisos e build de produção conferido.
- Terraform: `fmt`, `validate` e `plan` sem mudanças pendentes depois do deploy.
- Código, nomes de arquivos e rotas em inglês, com glossário PT→EN em `docs/glossary.md`; README e documentação de arquitetura em cada pasta.

4. Ética, governança e segurança

4.1 Uso responsável e limites do protótipo

- O sistema não faz diagnóstico. Todas as telas de análise mostram que o modelo é experimental, e existe uma página própria de limitações no site.
- Não há filtro que recuse imagens sem relação: uma foto qualquer também recebe uma das classes do treino, com qualquer confiança. O protótipo pressupõe um usuário que escolhe o módulo e o exame corretos. Essa limitação está documentada no site e no README.
- O Cérebro sempre responde um dos três tipos de tumor: ele não detecta ausência de tumor, e a tela de resultado diz isso.
- Na Pele, a queratose actínica ficou fora do treino; o modelo só vale para imagens dermatoscópicas, não para fotos comuns de celular.
- O resultado mostra todas as probabilidades, e não só a classe vencedora, para que o usuário veja quando o modelo está em dúvida.

4.2 Viés e vazamento de dados

- **Possível vazamento no Multi Cancer.** O dataset de Oral e Cérebro já vem aumentado (rotação, zoom, espelhamento, brilho) e não é dividido por paciente. Variações da mesma imagem podem cair no treino e no teste, e a acurácia alta pode medir memorização.
- **Divisão por lesão na Pele.** O treino separou os dados por `lesion_id`, para que fotos da mesma lesão não fiquem no treino e no teste. A separação não é por paciente.
- **Desbalanceamento.** No HAM10000, nevo é cerca de 67% das imagens e dermatofibroma cerca de 1%.
- **Tons de pele.** O desempenho em tons de pele pouco representados no dataset é desconhecido.
- **Atalhos.** Quando classes vêm de aparelhos ou fontes diferentes, a rede pode aprender o aparelho e não a doença.

4.3 Privacidade (LGPD)

- **Minimização:** a imagem é processada em memória e descartada. A tabela `predictions` guarda só usuário, modelo, classe, confiança, tempo e horário.
- **Consentimento:** antes de cada análise, o usuário confirma que o material é autorizado e desidentificado; sem isso, a análise não é liberada.
- **Isolamento:** cada usuário vê só o próprio histórico, porque o backend filtra pelo id do token, nunca por um parâmetro enviado pelo navegador.
- **Dados de cadastro:** só nome, e-mail e hash da senha. Não há dados de pacientes no banco nem seed com dados pessoais.

4.4 Segurança da aplicação

Ponto	Como foi tratado
Senhas	Hash BCrypt (Spring Security); a senha em texto não é gravada nem registrada em log.
Sessão	JWT assinado (HS256) com validade de 8 horas; segredo de pelo menos 32 caracteres, sem o qual o backend não sobe.
Enumeração de contas	Mesma mensagem de erro para e-mail inexistente e senha errada.
Comunicação entre serviços	O ai-service só aceita pedidos com o `X-Internal-Token`, comparado em tempo constante; o token é obrigatório para o serviço subir.
CORS	O backend aceita só a origem do site configurada em `CORS_ALLOWED_ORIGINS`.
Uploads	Limite de 10 MB no navegador, no backend e no ai-service; imagem ilegível é recusada antes de rodar o modelo e, na Pele, também a imagem com lado menor que 100 px.
Segredos	Nenhuma senha no código: tudo vem de variáveis de ambiente. Os arquivos `.env`, `terraform.tfvars` e `terraform.tfstate` ficam fora do git; variáveis do Terraform marcadas como `sensitive`.
Erros	Toda resposta de erro sai como `{"message": "..."}`, sem rastro de pilha nem detalhes internos.
Log e rastreabilidade	O backend registra cadastros, logins aceitos e recusados e cada análise (concluída ou não). O usuário aparece só pelo id: e-mail, senha, token e imagem nunca vão para o log.
Containers	Usuário sem privilégios; no compose, portas só em `127.0.0.1` e o ai-service sem porta exposta.

4.5 Licenças e créditos dos dados

Os dois datasets usam a licença CC BY-NC-SA 4.0: crédito obrigatório, sem uso comercial e mesma licença para derivados. O crédito aparece no site, no README e neste documento (seção 6). As imagens não estão no repositório. A licença é também o motivo de o projeto não estimar retorno comercial.

4.6 Riscos e mitigações

Risco	Mitigação atual	O que ainda falta
Usuário tratar o resultado como diagnóstico	Avisos em todas as telas de análise, página de limitações, inconclusivo e baixa certeza	Validação clínica antes de qualquer uso real
Imagem fora do domínio recebe classe com confiança alta	Limitação documentada; limiar de 80% em Oral e Cérebro	Filtro que recuse imagens incompatíveis
Acurácia inflada por vazamento	"Possível vazamento" escrito no site; divisão por lesão na Pele	Teste com imagens de outra fonte
Tentativas repetidas de login	Senhas com hash BCrypt, que torna cada tentativa mais lenta; logins recusados ficam registrados no log	Limite de tentativas por IP ou conta
Token roubado no navegador	Validade de 8 horas; React escapa os textos exibidos	Revogação de token e cookies `HttpOnly`
Perda do state do Terraform	Backup fora do repositório	State remoto com criptografia

5. Conclusão e trabalhos futuros

O PixoraMed entrega o caminho completo de um sistema de IA aplicado a imagens médicas: modelos treinados e avaliados com validação cruzada, um microsserviço de inferência em Docker, um backend com login seguro e regras de decisão testadas, uma interface que mostra resultado, histórico e métricas, e a infraestrutura descrita como código. Os três processos principais estão modelados em BPMN 2.0 e batem com o que o código faz.

O principal resultado, além do protótipo funcionando, é a leitura honesta dos números: acurácias acima de 95% na mesma fonte convivem com quedas fortes em imagens de outra origem, e o sistema foi desenhado para mostrar essa incerteza em vez de escondê-la.

Trabalhos futuros

- Testar Oral e Cérebro com imagens de outras fontes e publicar o resultado na página de treinos.
- Filtro que identifique o tipo de exame e recuse imagens sem relação antes da classificação.
- Calibrar as probabilidades de Oral e Cérebro, como já é feito na Pele.
- Dividir os dados por paciente e avaliar o desempenho por subgrupo (por exemplo, tom de pele).
- Limite de tentativas de login, revogação de token e state remoto do Terraform.

6. Referências

- Obuli Sai Naren. (2022). *Multi Cancer Dataset* [Data set]. Kaggle. https://doi.org/10.34740/KAGGLE/DSV/3415848
- Tschandl, P., Rosendahl, C. & Kittler, H. (2018). The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions. *Scientific Data*, 5, 180161. https://doi.org/10.1038/sdata.2018.161
- Creative Commons. Atribuição-NãoComercial-CompartilhaIgual 4.0 Internacional (CC BY-NC-SA 4.0). https://creativecommons.org/licenses/by-nc-sa/4.0/
- Object Management Group. Business Process Model and Notation (BPMN), versão 2.0. https://www.omg.org/spec/BPMN/2.0/
- Brasil. Lei nº 13.709, de 14 de agosto de 2018 (Lei Geral de Proteção de Dados Pessoais).

Repositório do sistema: github.com/joaopgf/PixoraMed. Notebooks de treino: github.com/joaopgf/Projeto-ML-Cancer.